

Reviewer Pack — Tropical Category Theory Bounds Frege Proof Depth

Entry 9e67dce52944 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 04:56:23 UTC

Tropical Category Theory Bounds Frege Proof Depth

Entry ID: 9e67dce52944 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 04:56:23 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory:
Frege Proof Complexity

Statement:

[‘For any Boolean formula F with n variables, the tropical category associated with its clause-indicator polynomial has a depth that is upper-bounded by some function of n , specifically: $\text{Depth}(\text{TropCat}(F)) \leq \alpha(n)$ where $\alpha(n) = O(\log^2(n))$.’, ‘For all instances F with Frege proof depth $d(F)$, the corresponding tropical category has a depth that satisfies: $\text{Depth}(\text{TropCat}(F)) \leq \beta * \log(d(F))$, for some constant β .’]

Rationale (proposer’s reasoning):

[‘Tropical geometry has been used to provide lower bounds for communication complexity and circuit complexity. By exploring the tropical category of clause-indicator polynomials, we may uncover new invariants that could shed light on proof depth, potentially leading to a better understanding of proof systems.’, ‘The categorical nature of tropical geometry offers a structured approach to analyzing combinatorial objects like Boolean formulas, which might reveal subtle connections between the geometric properties of these objects and their computational complexity.’]

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 91e78665aded6ec7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The tropical category depth of Boolean formulas F is compared against $\alpha(n) = O(\log^2(n))$ and $\beta * \log(d(F))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "Frege proof complexity" - "depth of tropical category" AND "Boolean formula" - "upper bound on depth" IN "tropical geometry" AND Frege proof complexity

Top relevant hits considered: - [s2:10.4230/LIPIcs.STACS.2025.8] Tropical Proof Systems: Between R(CP) and Resolution - [s2:1806.01835] Geometry and algorithms for upper triangular tropical matrix identities

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(b)
    augmented_matrix = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
        pivot = augmented_matrix[i][i]
```

```

        for j in range(i, n+1):
            augmented_matrix[i][j] /= pivot
        for j in range(n):
            if j != i:
                factor = augmented_matrix[j][i]
                for k in range(i, n+1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return [row[-1] for row in augmented_matrix]

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det

def is_full_rank(A):
    return determinant(A) != 0

def random_3cnf(n, m):
    variables = list(range(1, n+1))
    clauses = []
    for _ in range(m):
        clause = [random.choice([1, -1]) * random.choice(variables) for _ in range(3)]
        if len(set(clause)) == 3:
            clauses.append(clause)
    return clauses

def tropical_add(a, b):
    return max(a, b)

def tropical_multiply(a, b):
    return a + b

def tropical_negate(a):
    return -a

def tropical_zero():
    return float('-inf')

def tropical_one():
    return 0

def tropical_polynomial(clauses):
    n = len(clauses[0])
    poly = [[tropical_zero() for _ in range(n)] for _ in range(n)]
    for clause in clauses:
        for i, var in enumerate(clause):
            if var > 0:
                poly[i][var-1] = tropical_add(poly[i][var-1], tropical_one())
            else:
                poly[-i-1][-var-1] = tropical_add(poly[-i-1][-var-1], tropical_one())
    return poly

```

```

def tropical_category_depth(poly):
    n = len(poly)
    depth = 0
    while True:
        new_poly = [[tropical_zero() for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if poly[i][j] != tropical_zero():
                    for k in range(n):
                        new_poly[i][k] = tropical_add(new_poly[i][k], tropical_multiply(poly[i][j], tropical_zero()))
        if new_poly == poly:
            break
        poly = new_poly
        depth += 1
    return depth

def frege_proof_depth(clause_count):
    # Placeholder function to simulate Frege proof depth calculation
    return clause_count * 2

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = random.randint(2 * n, 4 * n)
        clauses = random_3cnf(n, m)
        poly = tropical_polynomial(clauses)

        if not is_full_rank(poly):
            return {
                "metric_name": "tropical_category_depth",
                "metric_value": None,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }

        depth = tropical_category_depth(poly)
        d_F = frege_proof_depth(m)

        results.append({
            "n": n,
            "m": m,
            "depth": depth,
            "d_F": d_F
        })

    alpha_n_values = [math.log2(n) ** 2 for n in n_values]
    beta = 0.5 # Placeholder value for beta

    mean_depth = sum(result["depth"] for result in results) / len(results)
    std_depth = math.sqrt(sum((result["depth"] - mean_depth) ** 2 for result in results) / len(results))

```

```

conjecture_holds = all(depth <= alpha_n and depth <= beta * math.log(d_F) for result in results)

counterexample = "" if conjecture_holds else "depth does not satisfy bounds"

return {
    "metric_name": "tropical_category_depth",
    "metric_value": mean_depth,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    all_results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        all_results.append(result)

    mean_depth = sum(result["metric_value"] for result in all_results if result["metric_value"] is not None) / len(all_results)
    std_depth = math.sqrt(sum((result["metric_value"] - mean_depth) ** 2 for result in all_results if result["metric_value"] is not None) / len(all_results))

    support_fraction = sum(1 for result in all_results if result["conjecture_holds"]) / len(all_results)

    if all(result["conjecture_holds"] for result in all_results):
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_depth} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in all_results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, all_results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"depth does not satisfy bounds\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8f6f234.py", line 176, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8f6f234.py", line 130, in run_trial
    poly = tropical_polynomial(clauses)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8f6f234.py", line 99, in tropical_polynomial
    poly[-i-1][-var-1] = tropical_add(poly[-i-1][-var-1], tropical_one())
                                ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying whether it supports or falsifies the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13278
2	preregistration	ollama_remote	glm4:latest	0	0	6107
3	novelty	ollama_remote	glm4:latest	0	0	4593
4	novelty	ollama_remote	glm4:latest	0	0	5614
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44801
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12237
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8312
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19897
9	judge	ollama_remote	glm4:latest	0	0	40812

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 155652 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/9e67dce52944.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9e67dce52944.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9e67dce52944.tar.gz (if generated)